

How to AI an AI for Almost Anything

Lecture 3 - LLMs as Context Optimizers

Paul Liang

Assistant Professor

MIT Media Lab & MIT EECS



<https://pliang279.github.io>

ppliang@mit.edu

 [@pliang279](https://twitter.com/pliang279)



Today's Lecture

- Recap on Prompt Optimization (5min)
- TextGrad Deep Dive (10min)
- LLM for Discovery (20min)
- FunSearch Breakdown (15min)
- Open Challenges (3min)

Recap on Prompt Optimization

- Context is everything the model reads before generating the next token. Prompt is what you give a model to guide its response.

Recap on Prompt Optimization

- Context is everything the model reads before generating the next token.
Prompt is what you give a model to guide its response.

types

developer

system prompt

instructions

tool definitions

examples

memory

user

user message

environment

files

search results

tool output

the model

its own thoughts and actions

uses

Inform

new information

Steer

instructions, role, format

Demonstrate

examples

Compute

thinking step by step

Correct

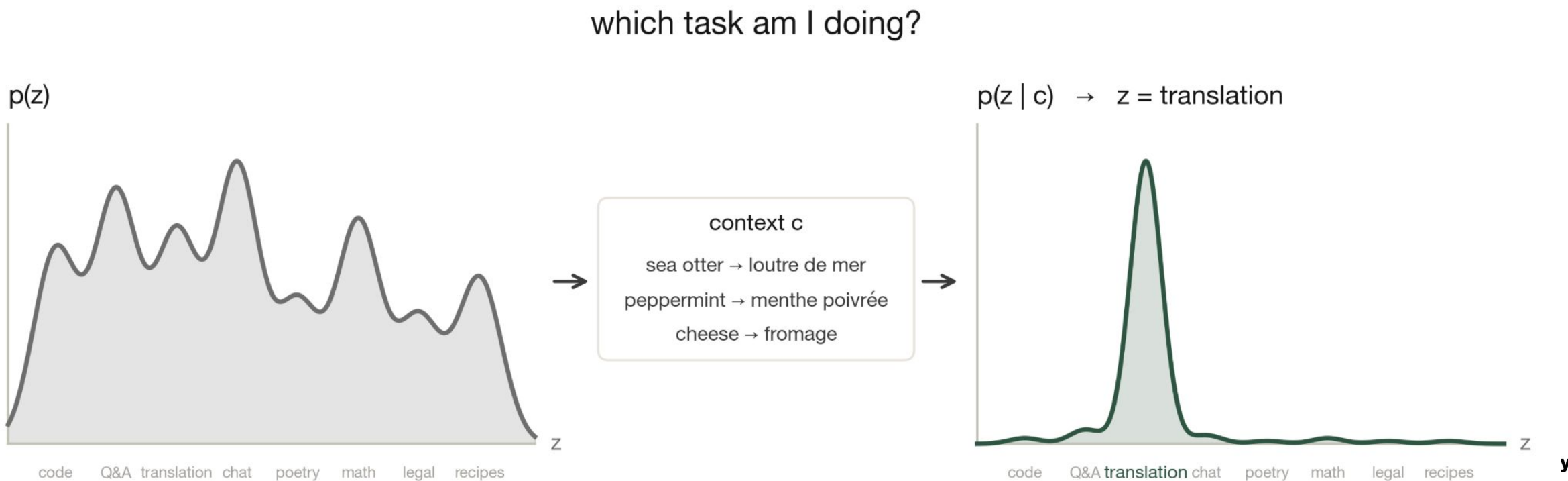
feedback from the last attempt

Recap on Prompt Optimization

- Context is everything the model reads before generating the next token. Prompt is what you give a model to guide its response.
- Prompt can shape model's behavior

Recap on Prompt Optimization

- Context is everything the model reads before generating the next token. Prompt is what you give a model to guide its response.
- Prompt can shape model's behavior

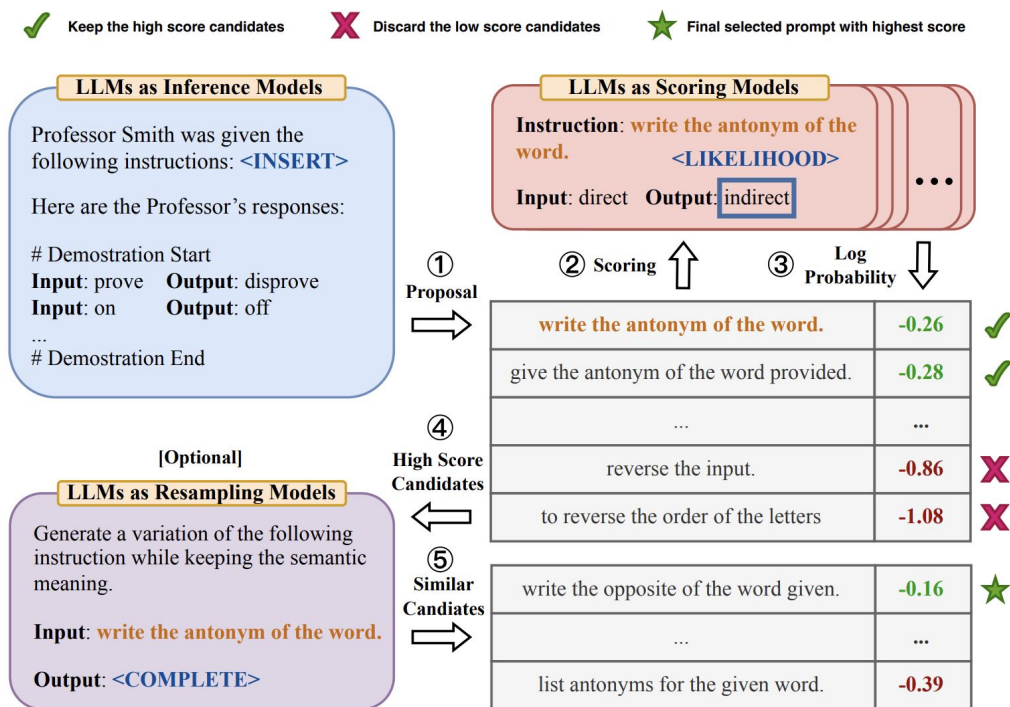


Recap on Prompt Optimization

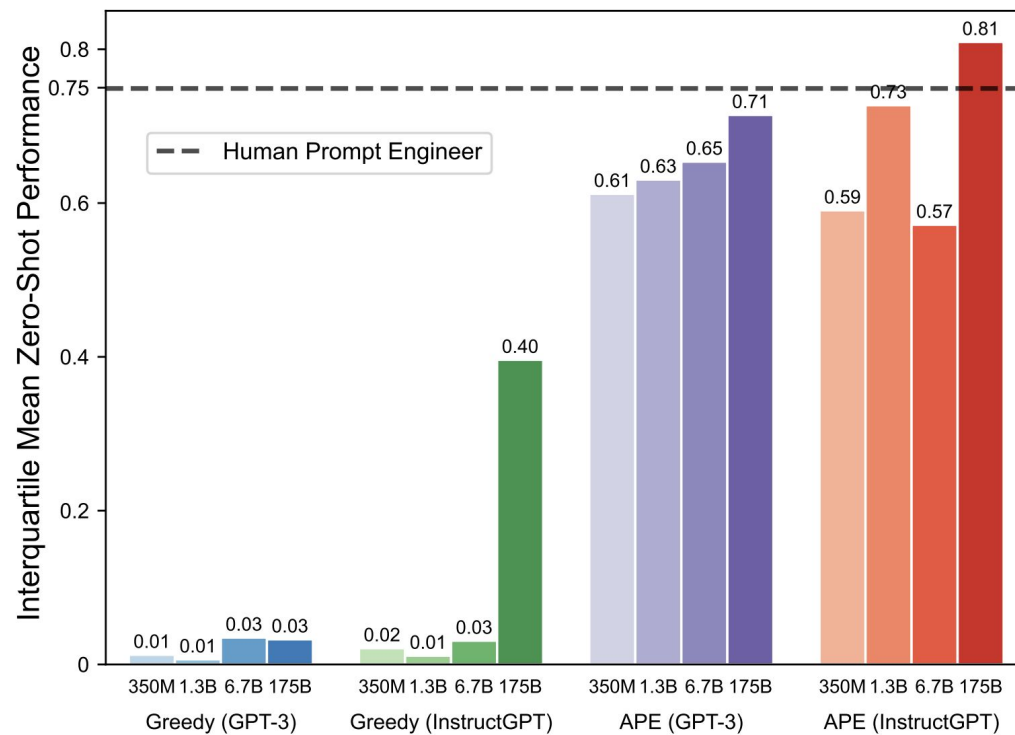
- Context is everything the model reads before generating the next token. Prompt is what you give a model to guide its response.
- Prompt can shape model's behavior
- LLMs can write high-quality prompts through iterative attempts

Recap on Prompt Optimization

Automated prompt engineer



(a) Automatic Prompt Engineer (APE) workflow



(b) Interquartile mean across 24 tasks

Recap on Prompt Optimization

- Context is everything the model reads before generating the next token. Prompt is what you give a model to guide its response.
- Prompt can shape model's behavior
- LLMs can write high-quality prompts through iterative attempts
- Towards Context Optimization: We can build an agentic system to update useful contexts everytime we solve a problem

Recap on Prompt Optimization

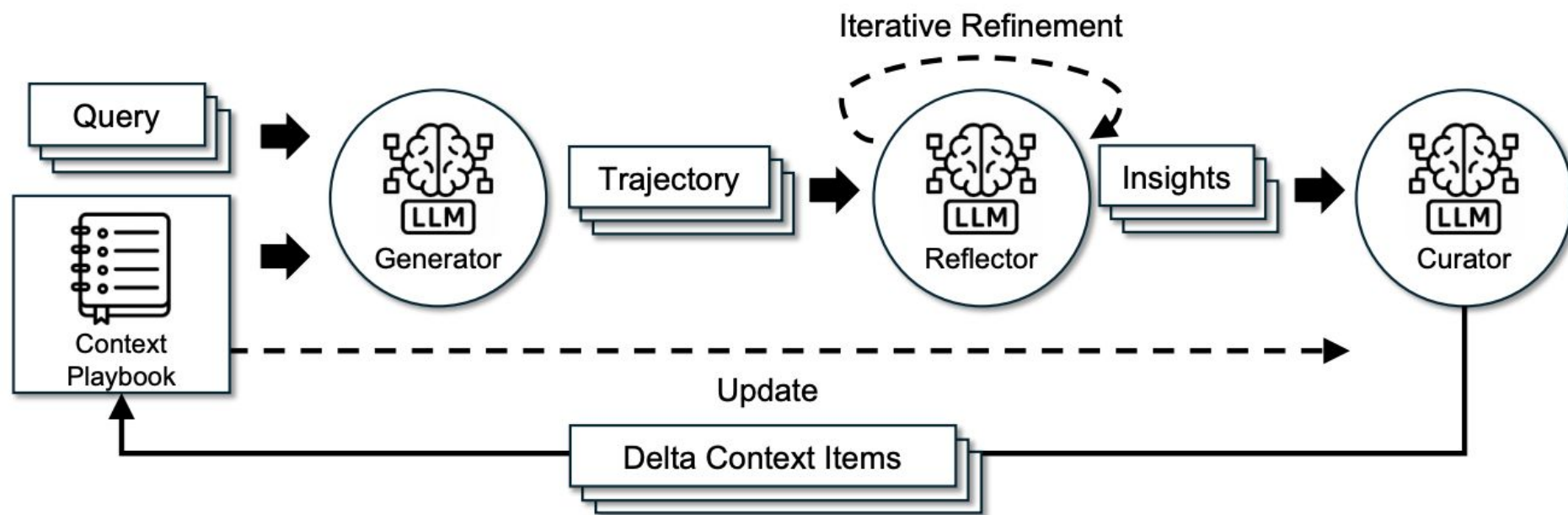


Figure 4: **The ACE Framework.** Inspired by Dynamic Cheatsheet, ACE adopts an agentic architecture with three specialized components: a Generator, a Reflector, and a Curator.

Recap on Prompt Optimization

The Generated ACE Playbook on AppWorld

STRATEGIES AND HARD RULES

[shr-00009]

When processing time-sensitive transactions involving specific relationships: always resolve identities from the correct source app (phone contacts), use proper datetime range comparisons instead of string matching, and verify all filtering criteria (relationship + time) are met before processing items. This ensures accurate identification and processing of the right transactions.

USEFUL CODE SNIPPETS AND TEMPLATES

[code-00013]

For efficient artist aggregation when processing songs, use defaultdict(list) to map song titles to artist names:

```
from collections import defaultdict
artist_map = defaultdict(list)
for song in songs:
    artist_map[song['title']].extend([artist['name'] for artist in song['artists']])
```

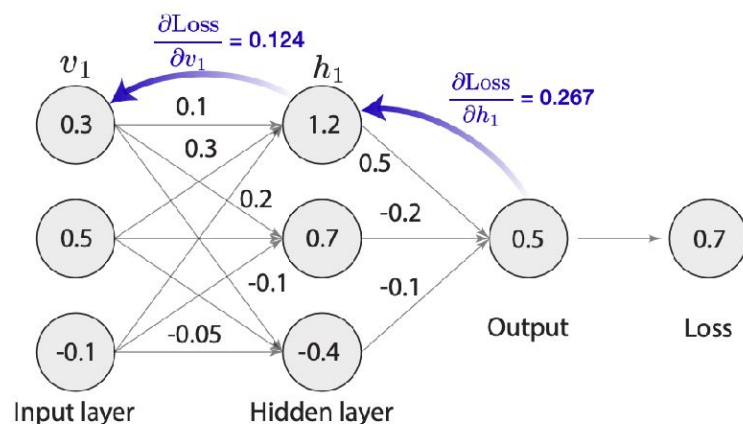
TROUBLESHOOTING AND PITFALLS

[ts-00003]

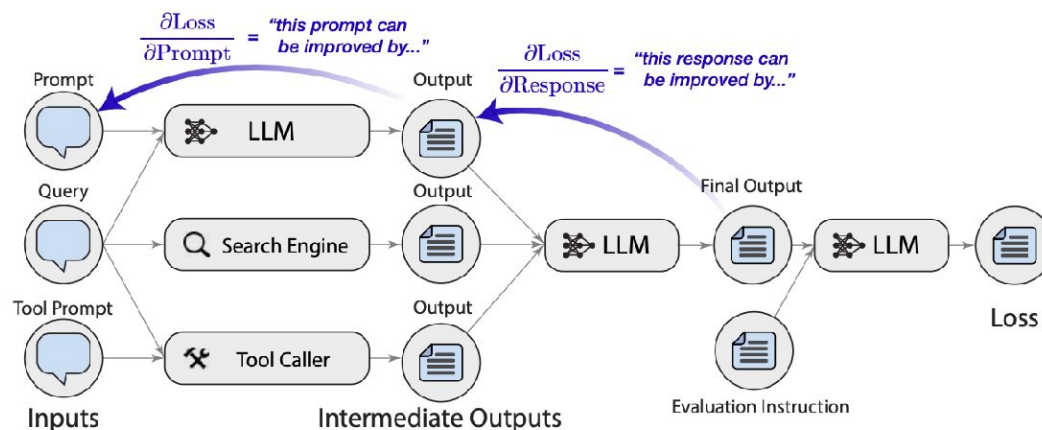
If authentication fails, troubleshoot systematically: try phone number instead of email as username, clean credentials from supervisor, check API documentation for correct parameters etc. Do not proceed with workarounds.

TextGrad Deep Dive (cont'd)

a Neural network and backpropagation using numerical gradients



b Blackbox AI systems and backpropagation using natural language 'gradients'



c 1 Analogy in abstractions

	Math	PyTorch	TextGrad
Input	x	<code>Tensor(image)</code>	<code>tg.Variable(article)</code>
Model	$\hat{y} = f_{\theta}(x)$	<code>ResNet50()</code>	<code>tg.BlackboxLLM("You are a summarizer.")</code>
Loss	$L(y, \hat{y}) = \sum_i y_i \log(\hat{y}_i)$	<code>CrossEntropyLoss()</code>	<code>tg.TextLoss("Rate the summary.")</code>
Optimizer	$\text{GD}(\theta, \frac{\partial L}{\partial \theta}) = \theta - \frac{\partial L}{\partial \theta}$	<code>SGD(list(model.parameters()))</code>	<code>tg.TGD(list(model.parameters()))</code>

2 Automatic differentiation

PyTorch and TextGrad share the same syntax for backpropagation and optimization.

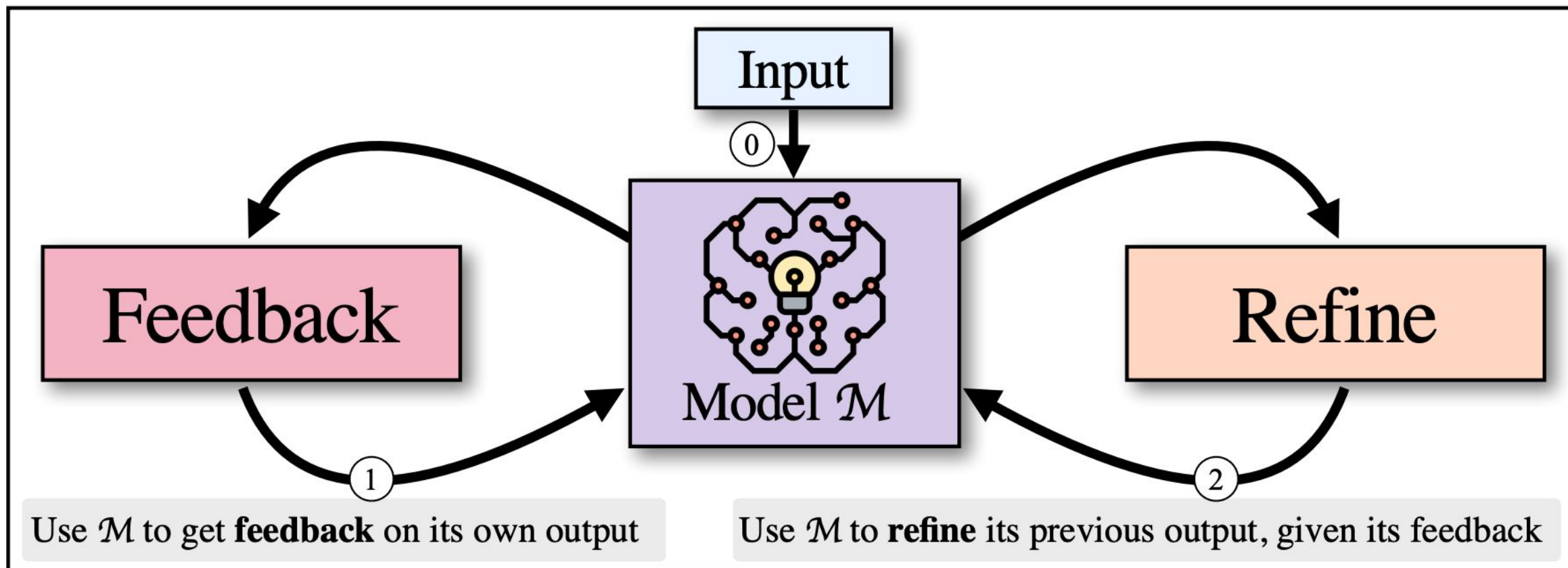
Forward pass
`loss = loss_fn(model(input))`

Backward pass
`loss.backward()`

Updating variable
`optimizer.step()`

We have used LLMs to optimize prompts. Can we use the same propose–evaluate–refine loop to optimize something else?

LLMs can optimize through self-refine



LLMs can optimize through self-refinement

(a) Dialogue: x, y_t

User: I am interested in playing Table tennis.

Response: I'm sure it's a great way to socialize, stay active

(b) FEEDBACK fb

Engaging: Provides no information about table tennis or how to play it.

User understanding: Lacks understanding of user's needs and state of mind.

(c) REFINE y_{t+1}

Response (refined): That's great to hear (...) ! It's a fun sport requiring quick reflexes and good hand-eye coordination. Have you played before, or are you looking to learn?

(d) Code optimization: x, y_t

```
Generate sum of 1, ..., N
def sum(n):
    res = 0
    for i in range(n+1):
        res += i
    return res
```

(e) FEEDBACK fb

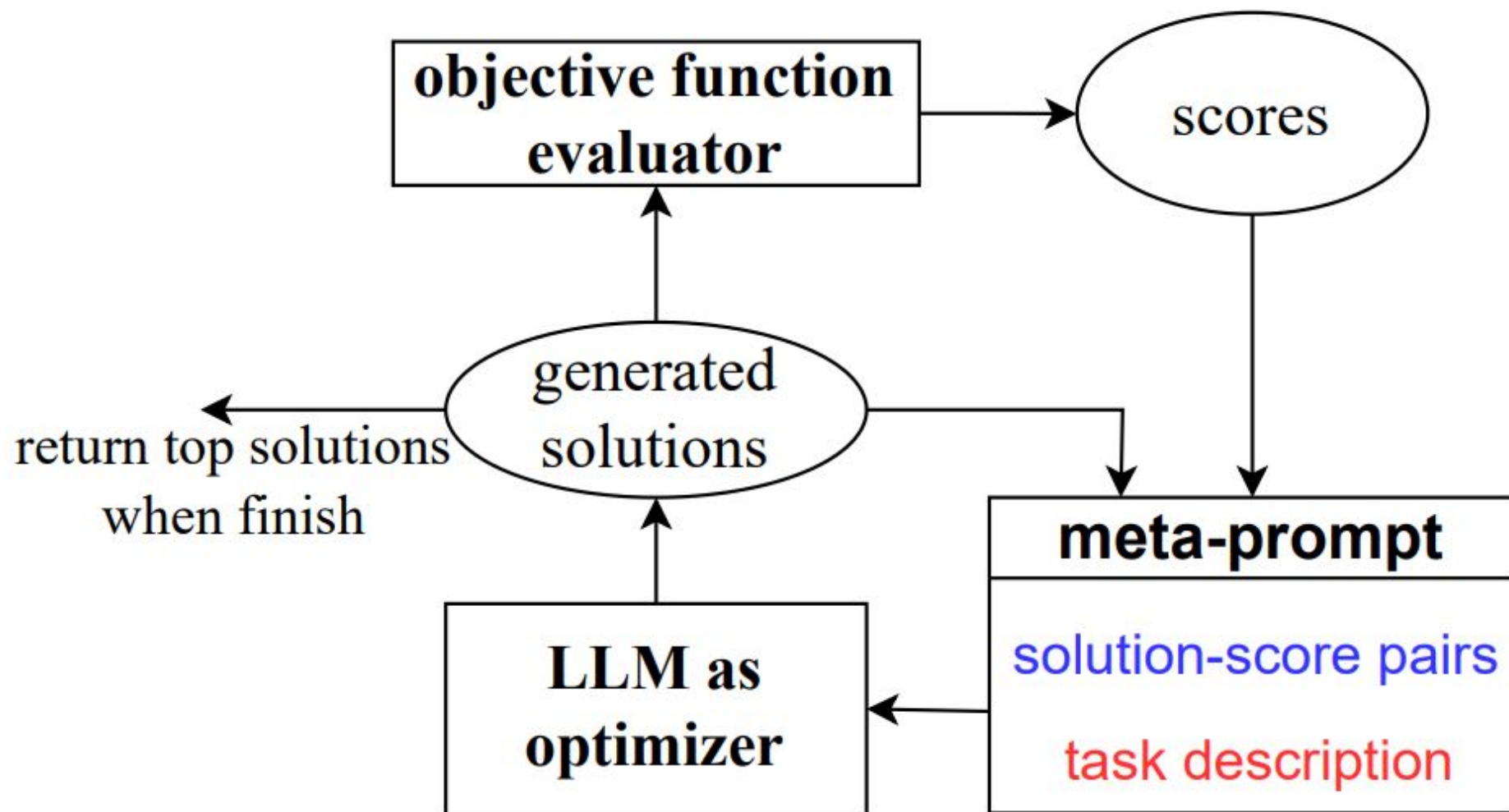
This code is slow as it uses brute force. A better approach is to use the formula ... $(n(n+1))/2$.

(f) REFINE y_{t+1}

Code (refined)

```
def sum_faster(n):
    return (n*(n+1))//2
```

LLMs can optimize using feedback from a scorer



LLMs can optimize using feedback from a scorer

You are given a list of points with coordinates below: (0): (-4, 5), (1): (17, 76), (2): (-9, 0), (3): (-31, -86), (4): (53, -35), (5): (26, 91), (6): (65, -33), (7): (26, 86), (8): (-13, -70), (9): (13, 79), (10): (-73, -86), (11): (-45, 93), (12): (74, 24), (13): (67, -42), (14): (87, 51), (15): (83, 94), (16): (-7, 52), (17): (-89, 47), (18): (0, -38), (19): (61, 58).

Below are some previous traces and their lengths. The traces are arranged in descending order based on their lengths, where lower values are better.

<trace> 0,13,3,16,19,2,17,5,4,7,18,8,1,9,6,14,11,15,10,12 </trace>

length:
2254

<trace> 0,18,4,11,9,7,14,17,12,15,10,5,19,3,13,16,1,6,8,2 </trace>

length:
2017

<trace> 0,11,4,13,6,10,8,17,12,15,3,5,19,2,1,18,14,7,16,9 </trace>

length:
1953

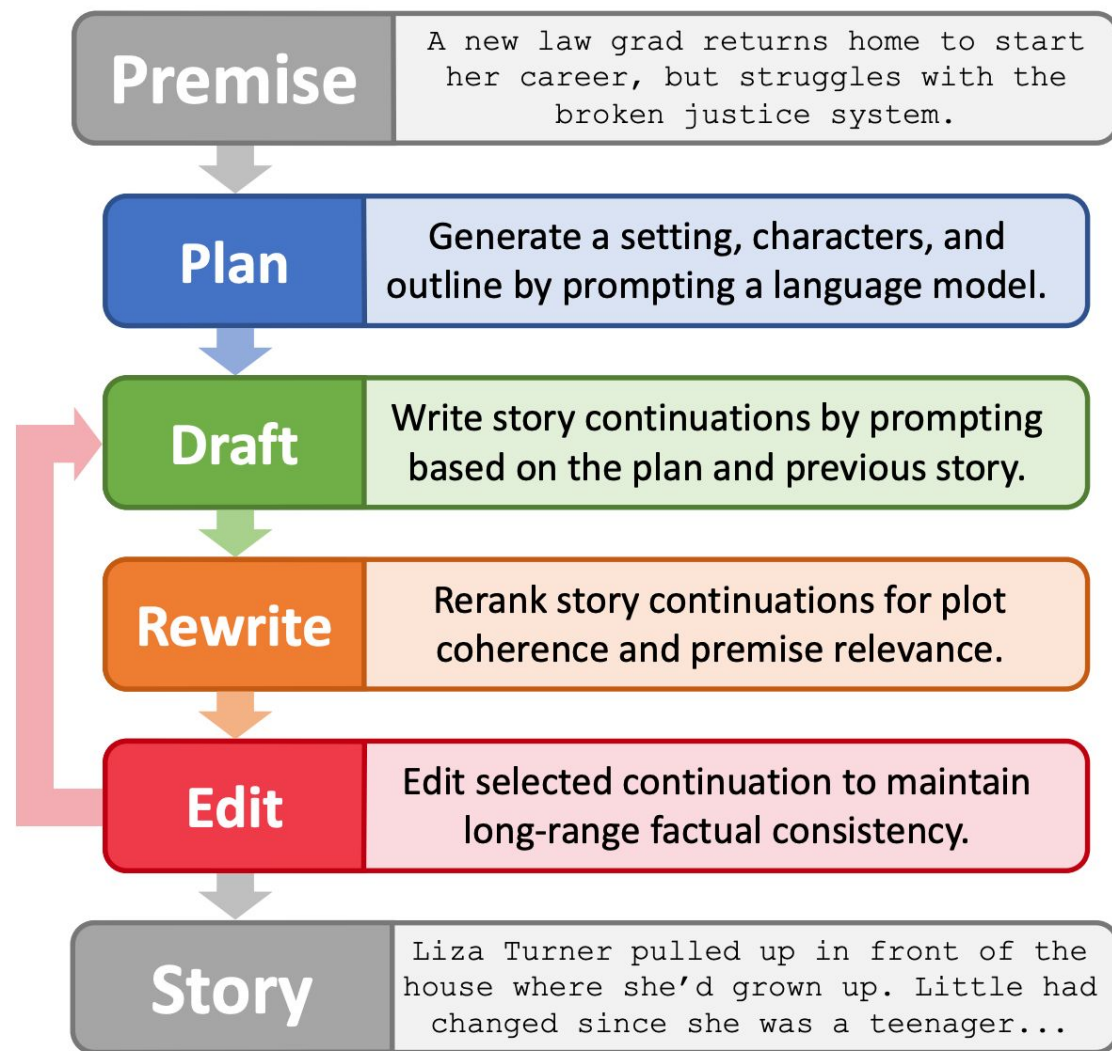
<trace> 0,10,4,18,6,8,7,16,14,11,2,15,9,1,5,19,13,12,17,3 </trace>

length:
1840

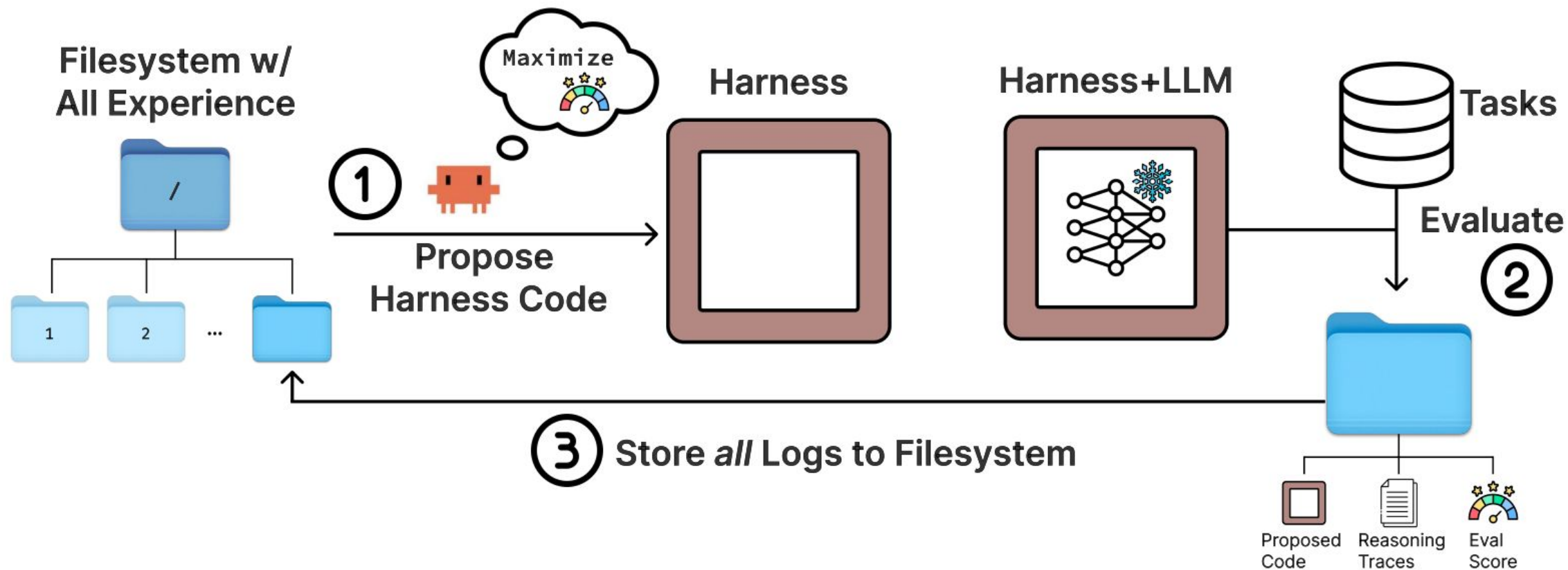
Give me a new trace that is different from all traces above, and has a length lower than any of the above. The trace should traverse all points exactly once. The trace should start with <trace> and end with </trace>.

Figure 20: An example of the meta-prompt for Traveling Salesman Problems with problem size $n = 20$. The blue text contains solution-score pairs; the orange text are meta-instructions.

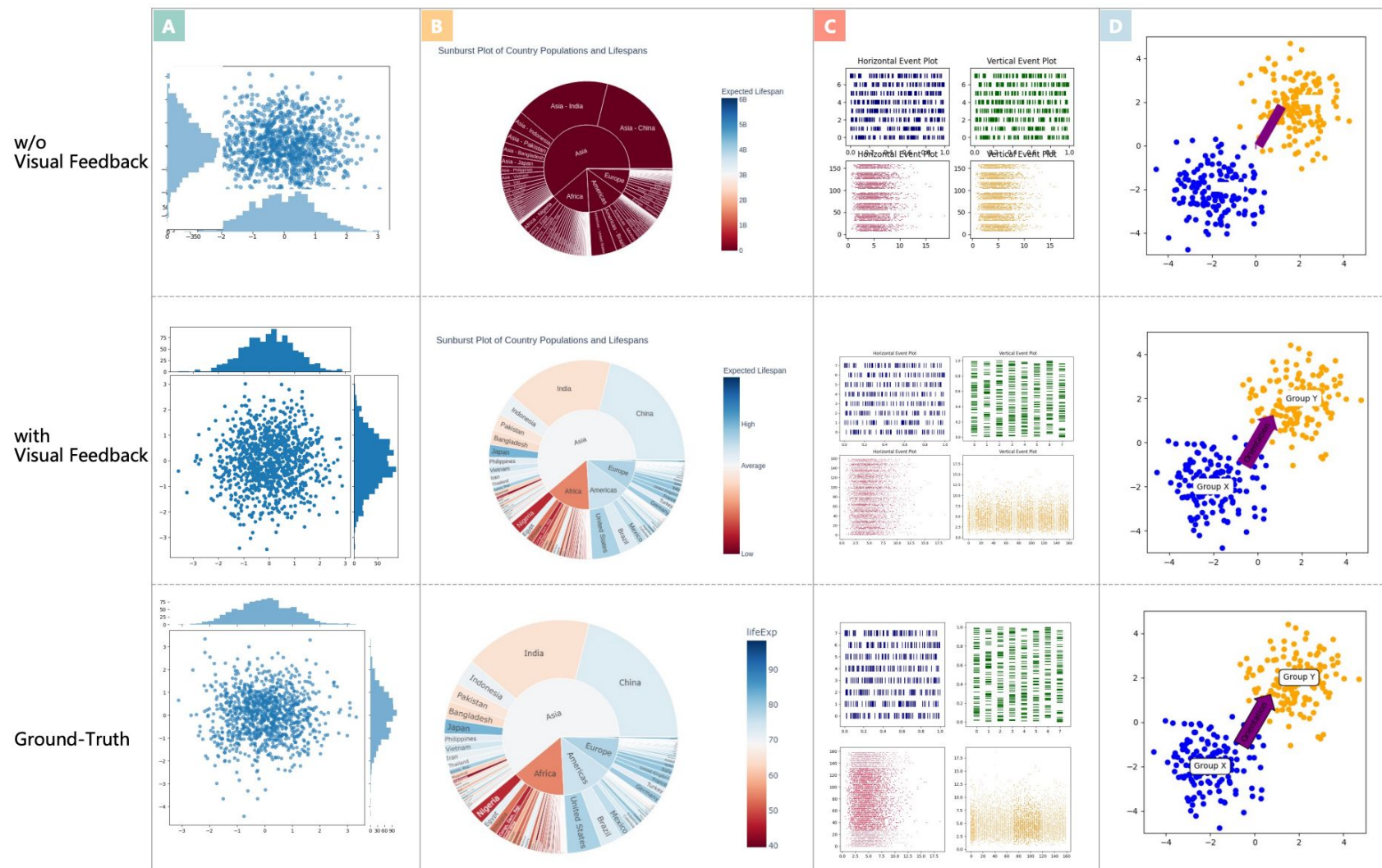
Optimize Writing



Optimize Agent Harness



Optimize Data Visualization



Optimize Image Generation

IDEA	T2I	Idea2Img	IDEA	T2I	Idea2Img	IDEA	T2I	Idea2Img
Object count 5 people sitting around a table drinking beer and eating buffalo wings			Knowledge an image of a hand holding an iphone. The image is used for illustrating how to take a screen shot on iphone			Visual design a logo suitable for a stylish hotel.		
Scene text a whole cake on the table with words Azure Research written on the cake			Prompt following a plate that has no bananas on it. there is a glass without orange juice next to it			Intended usage an image of a car perfect for a children's painting competition		
Concept customization & visual pointing photo of the object pointed by the blue arrow, and a brown corgi dog			Visual attribute referring Cartoon drawing of Mr Bean playing tennis, with the same cloth and pose as in the given image					
Style transfer painting of a corgi dog with style similar to this one in the image			Opposite style transfer painting of a corgi dog with style different from this one in the image					
Image manipulation a drawing with the background changed to a beach			Visual design & style A hand drawing of a room where people can sleep and study. Hand drawing shows the sketch and looks like the given image.					

Optimize Video Generation

Direct Prompting (DP)



Single-scene ([Polyak et al., 2025](#)): The person's forehead creased with worry as he listened to bad news.



Single-scene ([Polyak et al., 2025](#)): A spaceship entering hyperdrive, stars streaking past as it accelerates.

VISTA (Ours)



What about scientific discovery?

What can we achieve with LLM-driven search for solution optimization?

nature

[View](#)

[Explore content](#) ▾ [About the journal](#) ▾ [Publish with us](#) ▾

[nature](#) > [articles](#) > article

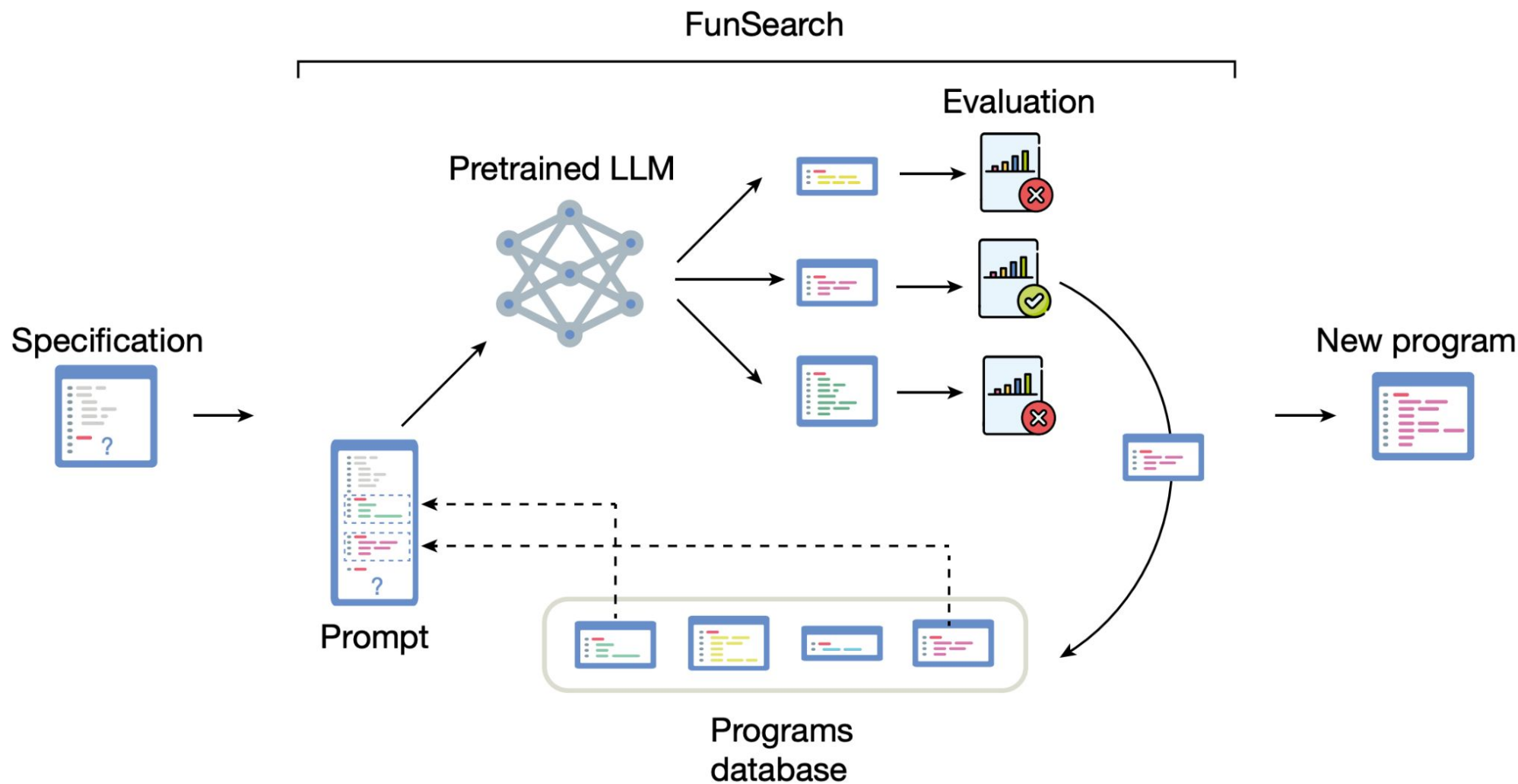
Article | [Open access](#) | Published: 14 December 2023

Mathematical discoveries from program search with large language models

[Bernardino Romera-Paredes](#) ✉, [Mohammadamin Barekatin](#), [Alexander Novikov](#), [Matej Balog](#), [M. Pawan Kumar](#), [Emilien Dupont](#), [Francisco J. R. Ruiz](#), [Jordan S. Ellenberg](#), [Pengming Wang](#), [Omar Fawzi](#), [Pushmeet Kohli](#) ✉ & [Alhussein Fawzi](#) ✉

1. Discovered new cap set constructions in eight dimensions, improving the record from 496 to 512.
2. Discovered bin-packing heuristics that outperformed standard baselines on tested benchmarks.

What can we achieve with LLM-driven search for solution optimization?



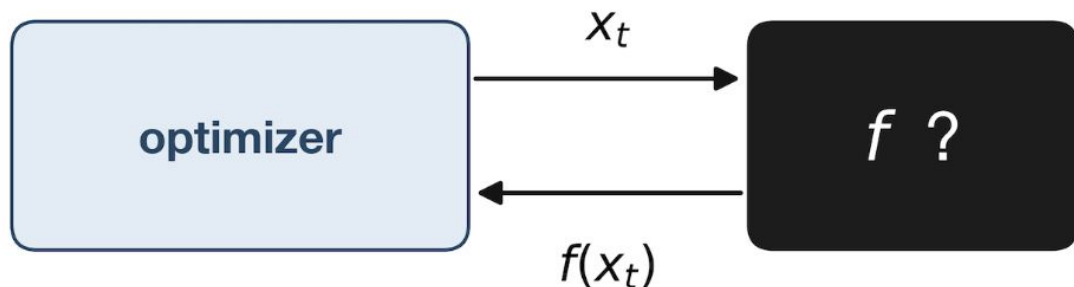
Background - Cap set problem & Bin-packing

1. Cap set problem: How many points can you select from $\{0,1,2\}^n$ without including three distinct points on a line?
 - a. In 8 dimensions, each point is an eight-entry vector, such as $(0,2,1,0,1,2,2,0)$.
 - b. Select as many of the 6,561 points in $\{0,1,2\}^8$ as possible, with no three distinct points on a line.
2. Bin-packing problem: How can we pack items of different sizes into the fewest bins, without exceeding any bin's capacity?
 - a. For example, suppose each bin has capacity 10, and the items have sizes 6, 5, 3, 2, 2, 2: A valid packing uses 3 bins: [6,3], [5,2,2], [2].
 - b. In online bin packing, items arrive one at a time. You must place each item immediately.

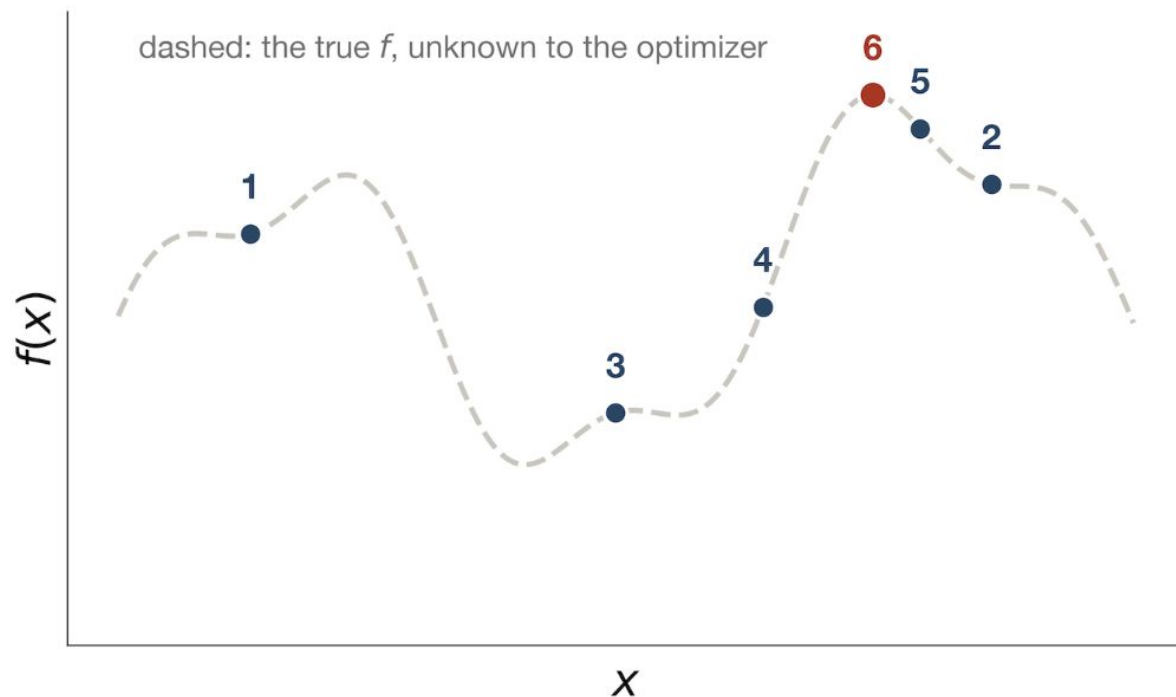
FunSearch Breakdown - Black Box Optimization

$$x^* = \arg \max_{x \in \mathcal{X}} f(x)$$

We can only evaluate f at the points we choose: $x_t \mapsto f(x_t)$



history $\{(x_1, f(x_1)), \dots, (x_t, f(x_t))\} \rightarrow x_{t+1}$



FunSearch Breakdown - Search Space

Search space: the set of candidates an optimizer can explore.

FunSearch searches over Python functions with given input and output format.

Cap sets: `priority(element, n)` ranks which vector to add next.

Bin packing: `heuristic(item, bins)` ranks where to place an item.

The output is a reusable program that constructs solutions.

FunSearch Breakdown - Search Space

$$p^* = \arg \max_p \text{score}(p)$$

Cap set

```
def priority(el: tuple[int, ...],  
            n: int) -> float:
```

```
    return 0.0                # seed
```

in: one vector $el \in \{0, 1, 2\}^n$ (n = dimension, fixed)

out: how much we want el in the set



evaluator: run the tests

score = size of the cap set

Online bin packing

```
def heuristic(item: float,  
             bins: np.ndarray) -> np.ndarray:
```

```
    return -(bins - item)    # seed
```

in: size of the new item, free space of each bin that fits it

out: one score per bin; the item goes to the highest



evaluator: run the tests

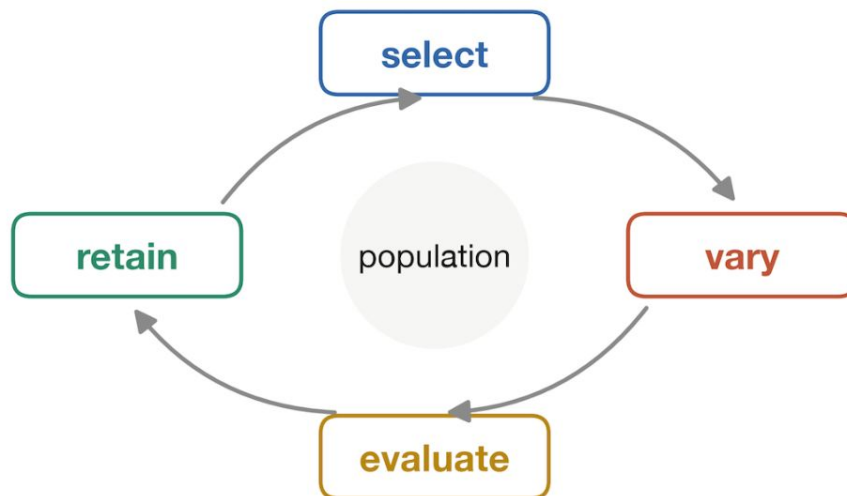
score = - number of bins used

FunSearch Breakdown - Evolutionary algorithms

```

population = initialize_programs()
evaluate(population)

for generation in range(budget):
    parents = select(population)
    children = vary(parents) # mutation / crossover
    evaluate(children)
    population = retain(population, children)
  
```



Operation	Purpose
Selection	Choose promising parents
Mutation	Introduce variations (e.g. change a constant, operator, or condition)
Crossover	Combine parents (e.g. swap parts of two programs)
Evaluation	Measure candidate quality
Survivor selection	Decide what to keep

Selection

$$P(i) = \frac{e^{f_i/T}}{\sum_j e^{f_j/T}}$$

small T : exploit large T : explore

Survivors

keep the best N of
parents + children

FunSearch Breakdown - Evolutionary algorithms

Evolutionary algorithms improve a population through repeated selection, variation, and evaluation.

FunSearch uses an LLM to generate variations of sampled programs.

```

"""Finds large cap sets."""
import numpy as np
import utils_capset

def priority_v0(element, n):
    """Returns the priority with which we want to add `element` to the cap set."""
    #####
    # Code from lowest-scoring sampled program.
    return ...
    #####

def priority_v1(element, n):
    """Improved version of `priority_v0`."""
    #####
    # Code from highest-scoring sampled program.
    return ...
    #####

def priority_v2(element, n):
    """Improved version of `priority_v1`."""

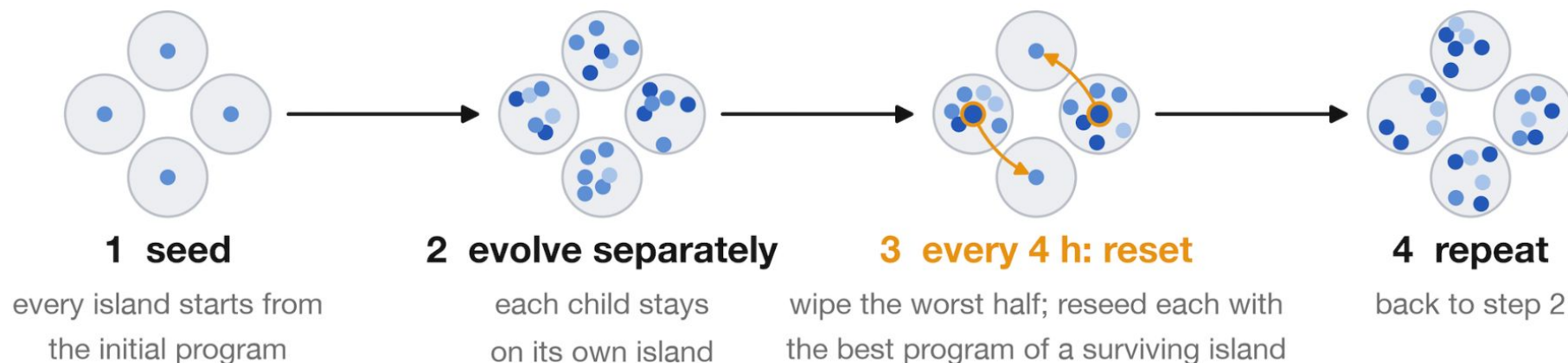
```

FunSearch Breakdown - Programs Pool & Exploration

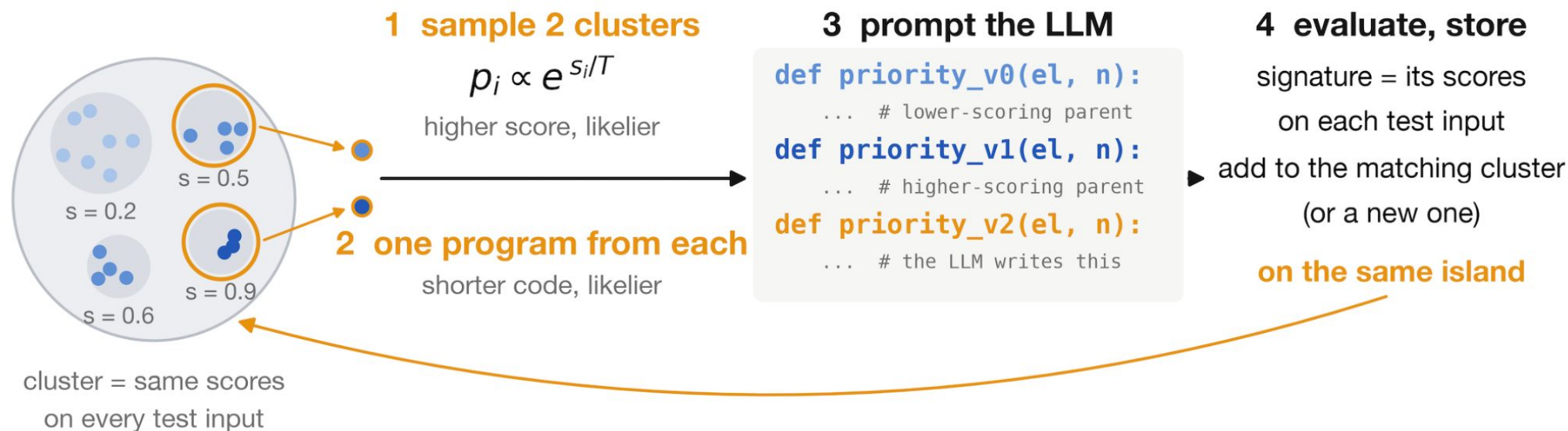
1. Initialize each island with a seed program
2. Grow the population within each island
 - a. Sample parents from the existing programs in this island
 - b. Generate a child program and add it to the same island
3. Remove the lower performing islands

A Across islands (10 islands; 4 shown)

● low score ● high score ○ sampled / copied



B One iteration (on an island picked uniformly at random)



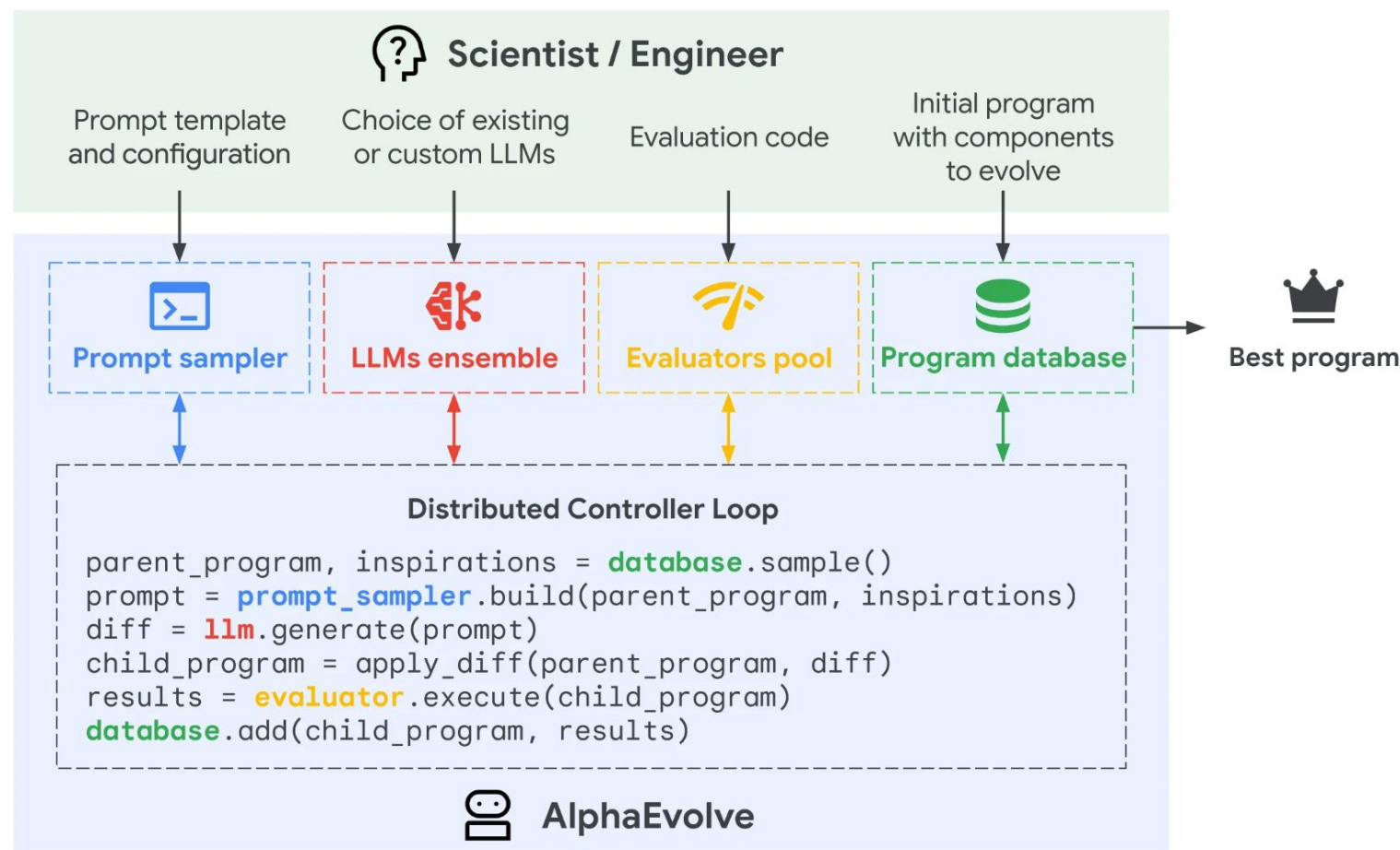
AlphaEvolve

Search scope:
Single-function →
Full-program evolution

Edit mode:
SEARCH/REPLACE diffs

Models: Stronger LLMs with
richer context and feedback

Objectives: Single-metric →
Multi-metric optimization

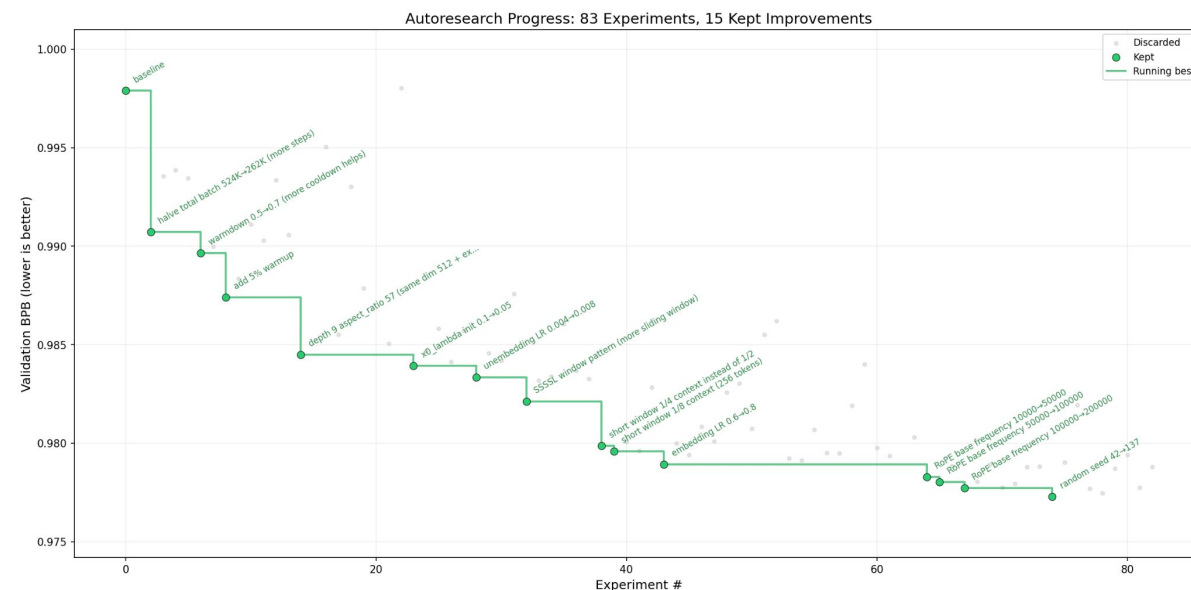


Autoresearch

PROGRAM.MD, "THE EXPERIMENT LOOP"

LOOP FOREVER:

1. Look at the git state
2. Tune `train.py` with an experimental idea
3. git commit
4. Run: `uv run train.py > run.log`
5. `grep "^val_bpb:" run.log`
6. If empty, the run crashed; read the trace
7. Record the results in the tsv
8. If val_bpb improved, keep the commit
9. If equal or worse, git reset

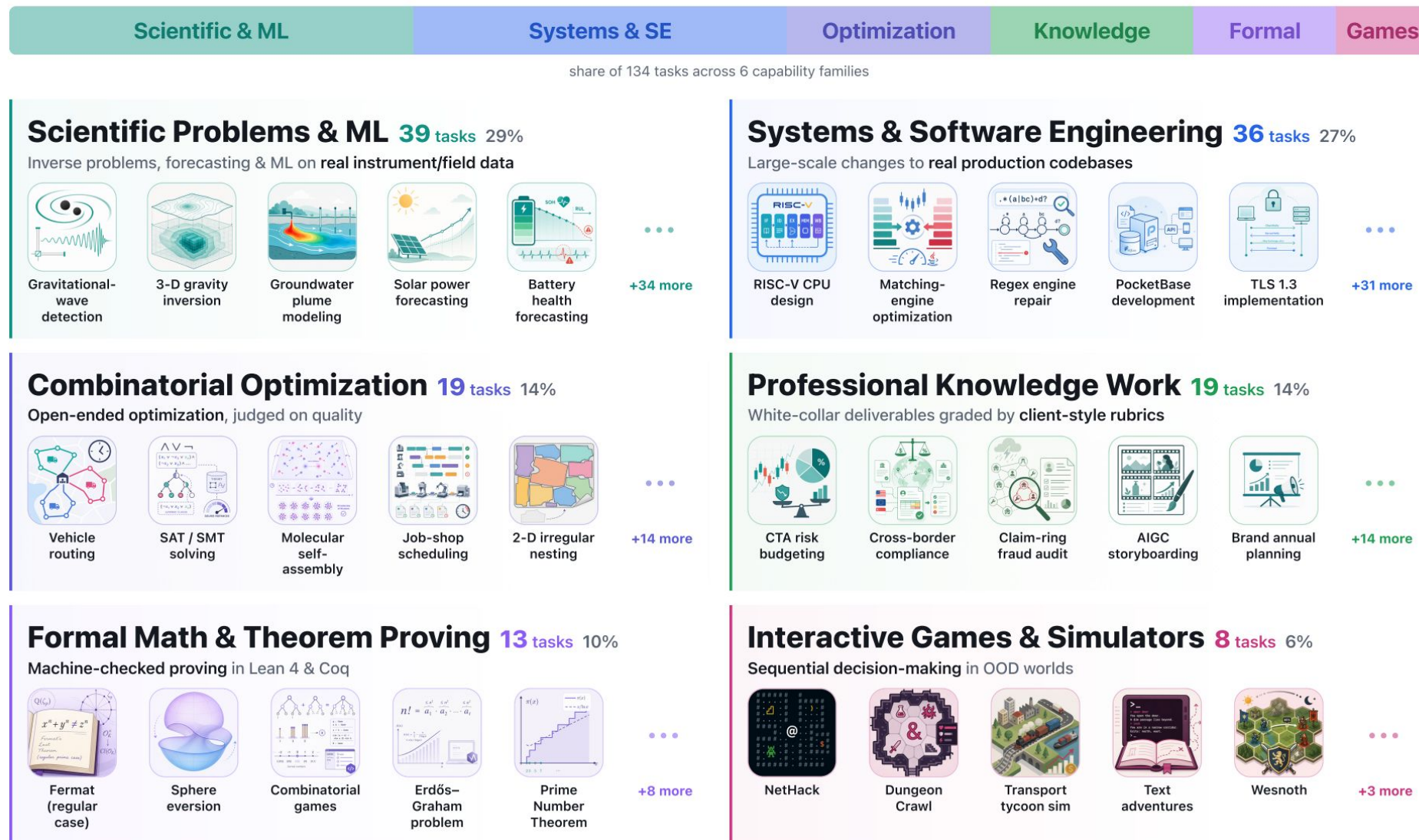


An AI agent follows program.md to propose ideas, edit code, and run experiments that improve language model pretraining under a fixed compute budget (5 minutes per experiment, ~100 experiments one night)

What other problems can you “autoresearch”?

EdgeBench: 134 real-world, day-long tasks across 6 capability families

Agent runtime ≥ 12 h per task · recorded human expert effort mean **57.2 h** · every task provides **rich, real-world feedback** for continuously learning from environment

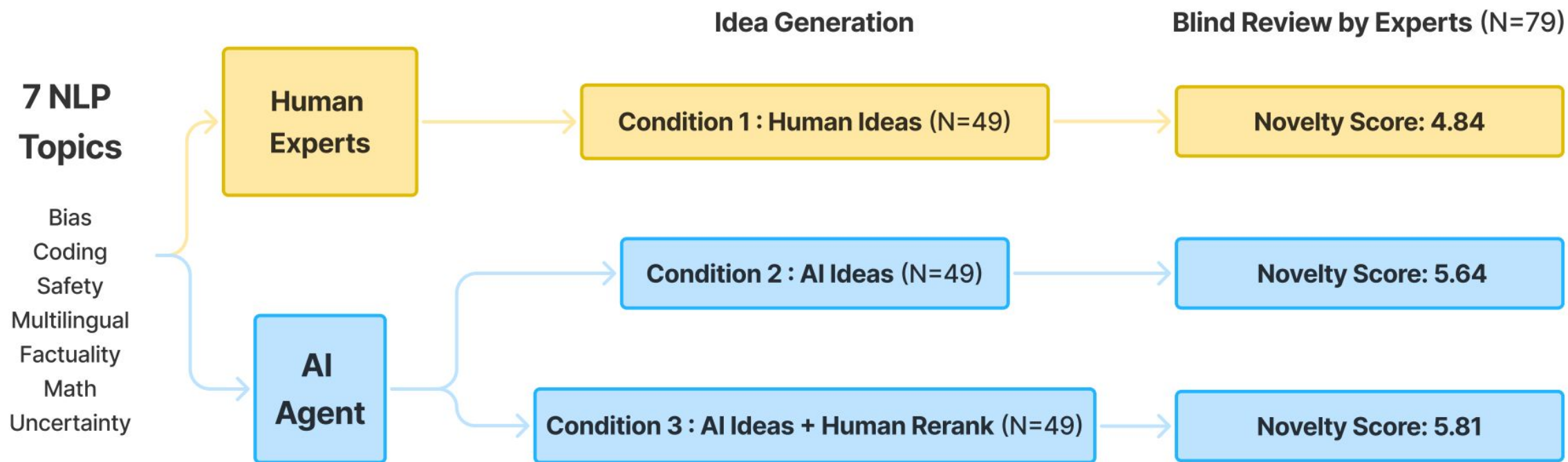


OpenEvolve Deep Dive - HW1 preview

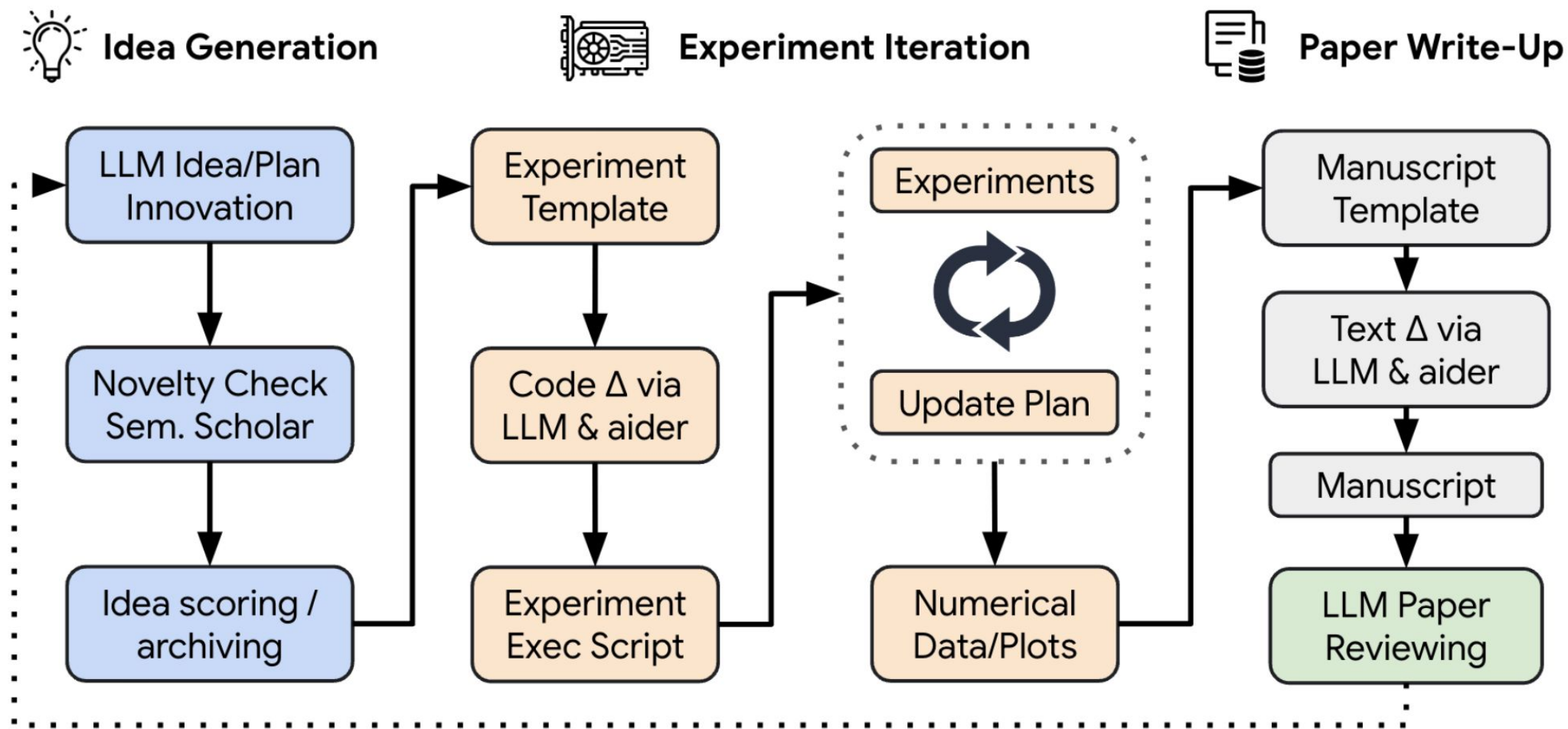
Many discoveries don't come with a numerical grader

- Biology: CRISPR gene editing
- Physics: General relativity
- Chemistry: Discovery of penicillin
- Astronomy: Dark matter
- Geology: Plate tectonics
- Medicine: mRNA vaccines
- Computer Science: Backpropagation

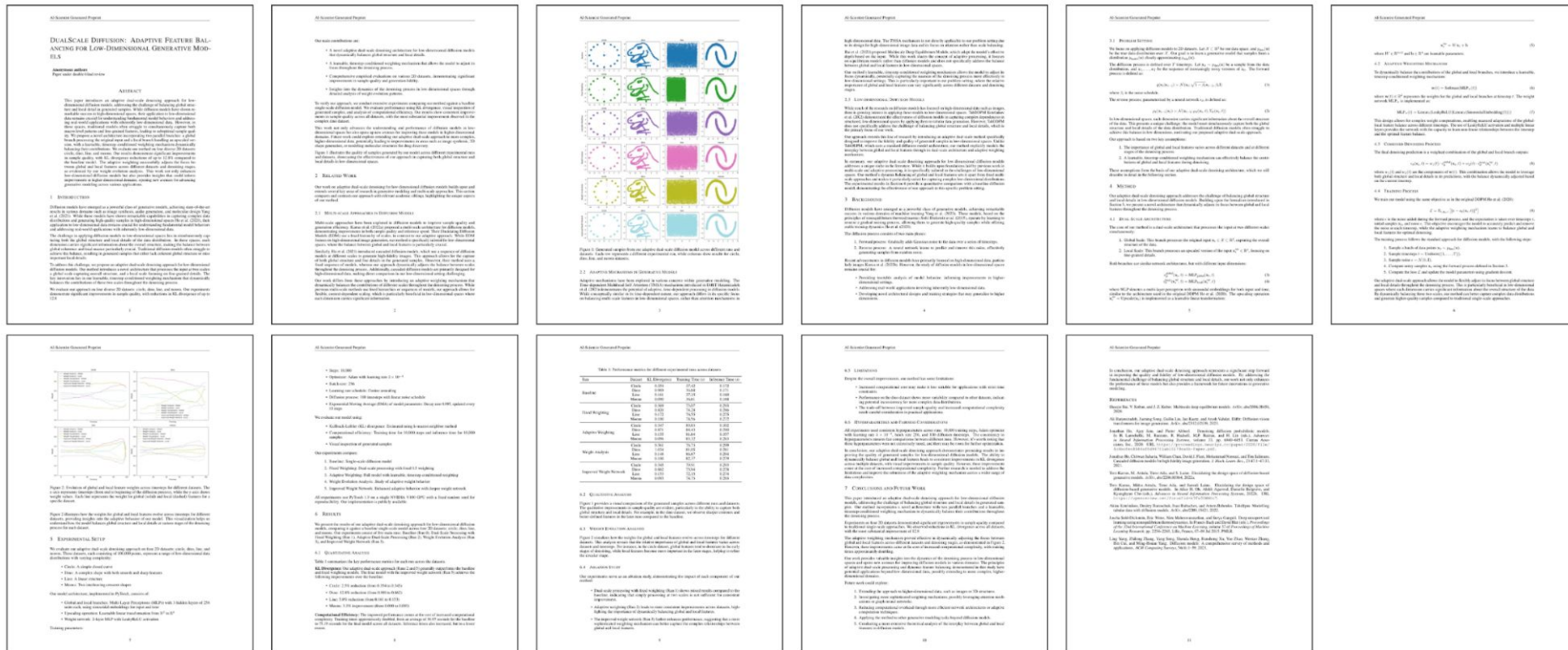
Can LLMs Generate Novel Research Ideas? A Large-Scale Human Study with 100+ NLP Researchers



AI Scientist



AI Scientist



An example paper, “Adaptive Dual-Scale Denoising” generated by The AI Scientist. The full paper can be viewed [here](#). While containing some flaws (e.g. a slightly unconvincing interpretation of why its method is successful), the paper proposes an interesting new direction that displays good empirical results in experiments The AI Scientist itself conducted and peer reviewed. More examples of generated papers are below.

Verification is important for discovery

DeepSeek-Prover-V2: Advancing Formal Mathematical Reasoning via Reinforcement Learning for Subgoal Decomposition

Z.Z. Ren*, Zhihong Shao*, Junxiao Song*, Huajian Xin[†], Haocheng Wang[†], Wanjia Zhao[†], Liyue Zhang, Zhe Fu
Qihao Zhu, Dejian Yang, Z.F. Wu, Zhibin Gou, Shirong Ma, Hongxuan Tang, Yuxuan Liu, Wenjun Gao
Daya Guo, Chong Ruan

DeepSeek-AI

- DeepSeek-V3 → decomposes the original theorem into subgoals
- DeepSeek-Prover-V2-7B → attempts to prove those subgoals in Lean

Verification is important for discovery



Show that for any integer $n \geq 4$, we have $n^2 \leq n!$.

```
theorem induction_ineq_nsqlifactn (n : ℕ) (h₀ : 4 ≤ n) : n ^ 2 ≤ n ! := sorry
```

Understanding the Problem
We need to show that for any integer $n \geq 4$, the inequality $n^2 \leq n!$ holds. Here, $n!$ denotes the factorial of n , which is the product of all positive integers up to n .

Proof Sketch
To prove this formally, we can use mathematical induction. Here's how the induction would work:
1. **Base Case:** Verify the inequality for $n = 4$.
2. **Inductive Step:** Assume the inequality holds for some $k = 4$, i.e., $k^2 \leq k!$. Then show that it holds for $k + 1$, i.e., $(k + 1)^2 \leq (k + 1)!$.

🧠 chain-of-thought by DeepSeek-V3

Lean 4 Proof Structure

```
theorem induction_ineq_nsqlifactn (n : ℕ) (h₀ : 4 ≤ n) : n ^ 2 ≤ n ! := by
  have base_case : 4 ^ 2 ≤ 4 ! := by
    sorry
  have inductive_step : ∀ k ≥ 4, k ^ 2 ≤ k ! → (k + 1) ^ 2 ≤ (k + 1) ! := by
    sorry
  have final_proof : ∀ n ≥ 4, n ^ 2 ≤ n ! := by
    sorry
    sorry
```



Recursively Solve Subgoals by DeepSeek-Prover-V2-7B

```
lemma inductive_step (n : ℕ) (h₀ : 4 ≤ n) (base_case : 4 ^ 2 ≤ 4 ! )
  : ∀ k ≥ 4, k ^ 2 ≤ k ! → (k + 1) ^ 2 ≤ (k + 1) ! := by
  sorry
```

Synthesize into Complete Formal Proofs

```
theorem induction_ineq_nsqlifactn (n : ℕ) (h₀ : 4 ≤ n) : n ^ 2 ≤ n ! := by
  have base_case : 4 ^ 2 ≤ 4 ! := by
    simp [Nat.factorial]
  have inductive_step : ∀ k ≥ 4, k ^ 2 ≤ k ! → (k + 1) ^ 2 ≤ (k + 1) ! := by
    intro k h₁ h₂
    simp_all [Nat.factorial]
    nlinarith
  have final_proof : ∀ n ≥ 4, n ^ 2 ≤ n ! := by
    intro n hn
    induction' hn with k hk
    case refl => exact base_case
    case step =>
      apply inductive_step k hk
      exact by assumption
    apply final_proof
    exact h₀
```

Figure 2 | Overview of the cold-start data collection process employed by DeepSeek-Prover-V2. We first prompt DeepSeek-V3 to generate a natural-language proof sketch while simultaneously formalizing it into a Lean statement with `sorry` placeholders for omitted proof details. A 7B prover model then recursively solves the decomposed subgoals. By combining these subgoal proofs, we construct a complete formal proof for the original complex problem. This composed proof is appended to DeepSeek-V3's original chain-of-thought, creating high-quality cold-start training data for formal mathematical reasoning.

Verification is important for discovery



Blog: <https://www.anthropic.com/research/formalizing-fermats-last-theorem>

Verification is important for discovery

Multi-agent = parallel test-time compute

Open Challenges

- Distribution shift from ultra-long contexts

How does model behavior change as context length grows, and how robust are optimization/discovery methods under these induced distribution shifts?

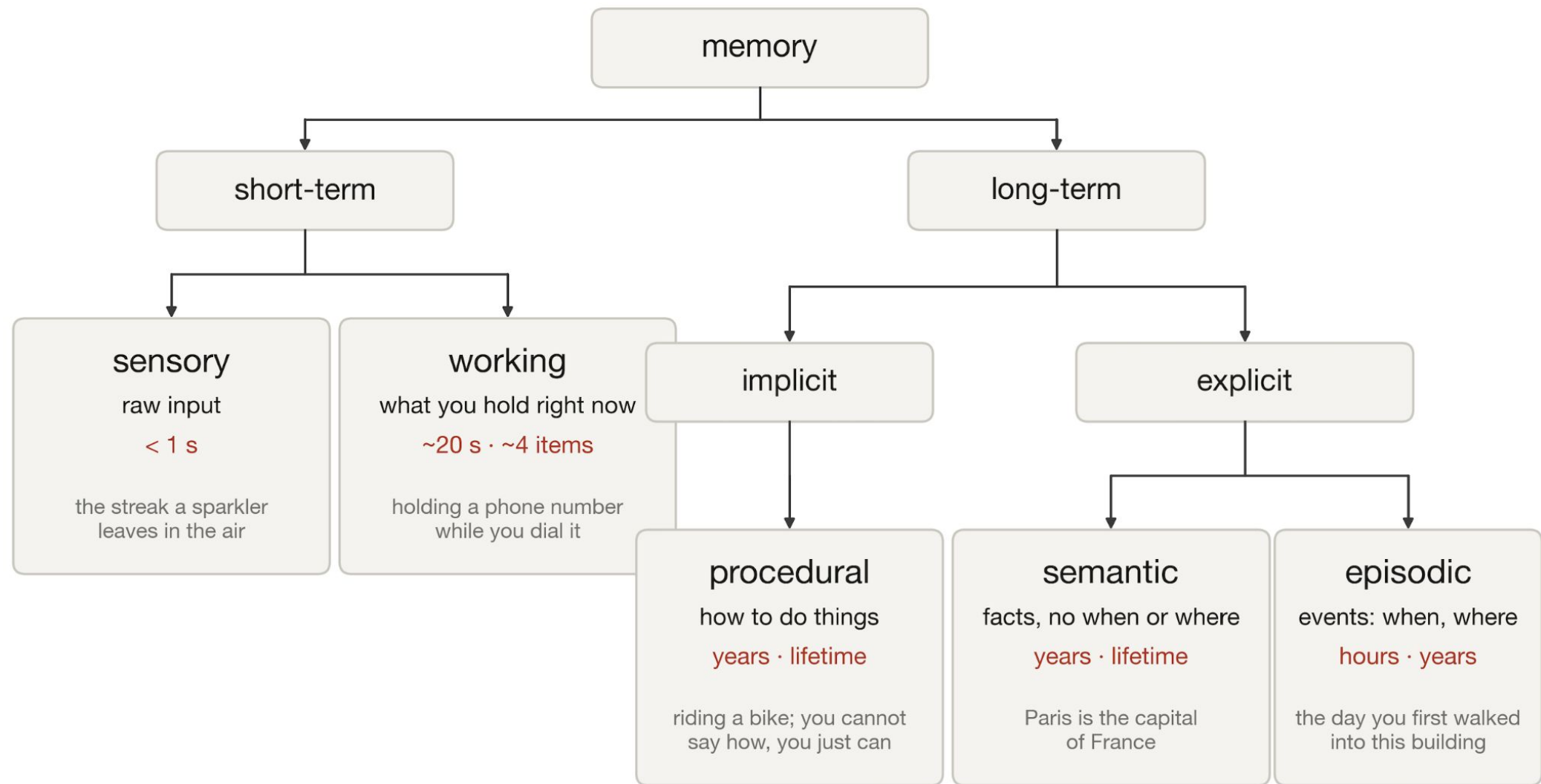
- Genuine discovery beyond verifiable domains

Can LLMs generate genuinely novel discoveries when correctness or progress cannot be easily verified by an automated evaluator?

- Faster Adaptation

Can models learn useful design principles from fewer experiments, enabling better decisions and designs with a smaller experimental budget?

Next: From Memory to Harness



Assignments for This Coming Week

We will notify you about your enrollment status this week.

- For-credit enrollment will be capped at 20-30
- Listeners are welcome to join lectures, discussions and course projects if they are dedicated, but there will be limited space and resources

Project preference form will be out this week:

- Tentative list of teammates
- Tentative directions and tasks